LAPORAN PRAKTIKUM

INTERNET OF THINGS II



Disusun Oleh:

Shavina Naura Putri

25782055

Dosen Pengampu:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

7.6 Latihan

- **Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):**
  Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks **mqtt**, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.
  i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?
  ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

  Jawaban:

  1. Mengapa penulisan localhost di dalam kontainer Docker menyebabkan kegagalan koneksi (*Connection Refused*)? Di dalam ekosistem Docker, setiap kontainer berjalan sebagai lingkungan sistem operasi yang terisolasi secara mandiri. Jika Anda menuliskan localhost (atau 127.0.0.1) di dalam kontainer Node-RED, sistem akan mengartikannya sebagai "perangkat lokal kontainer Node-RED itu sendiri", bukan laptop induk (*host*) ataupun kontainer Mosquitto. Karena di dalam kontainer Node-RED tidak ada layanan Mosquitto yang aktif pada port 1883, maka koneksi akan langsung ditolak (*Connection Refused*).

  2. Bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja? Ketika kontainer-kontainer didefinisikan bersama dalam berkas docker-compose.yml, Docker secara otomatis memasukkannya ke dalam satu jaringan *custom bridge* internal yang sama. Docker memiliki layanan *Embedded DNS Server* bawaan yang memetakan nama servis (*service name*, seperti mqtt) langsung ke alamat IP internal kontainer tersebut (mqtt-broker). Oleh karena itu, kontainer Node-RED dapat mengenali dan meneruskan paket data ke broker Mosquitto secara mulus hanya dengan memanggil nama layanannya (mqtt).

- **Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):**
  Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.
  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?
  ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

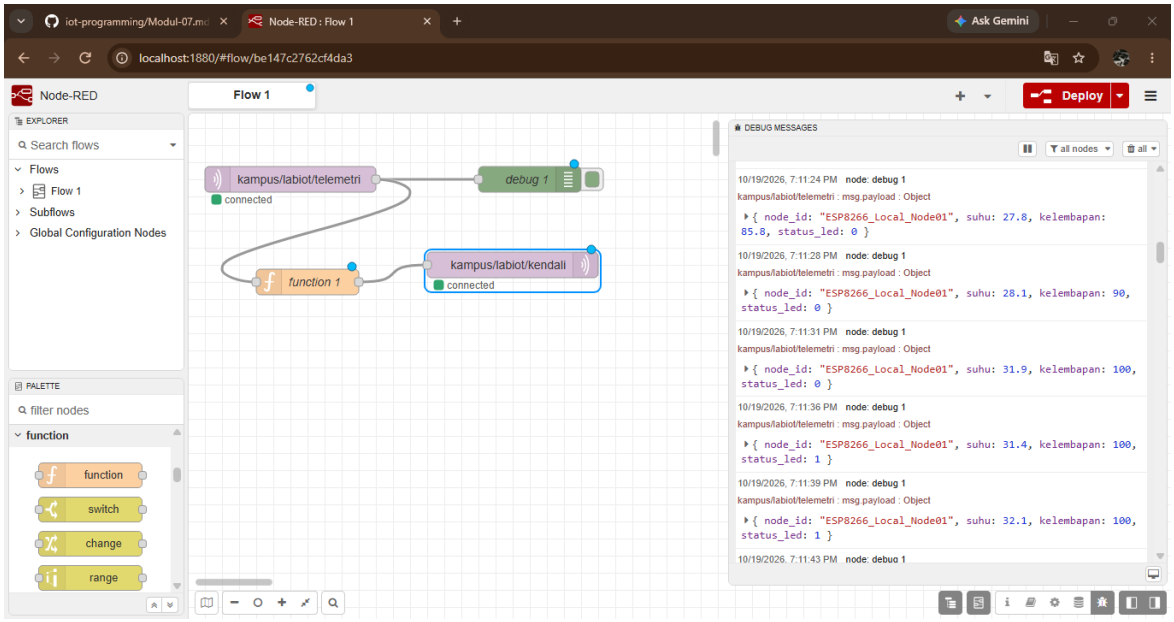
  Jawaban:

  1. **Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto?** Kedua perangkat berada pada segmen jaringan (*subnet*) atau *domain broadcast* yang berbeda secara total. NodeMCU berada di subnet 192.168.43.x (biasanya berasal dari *hotspot* seluler HP), sementara laptop berada di subnet 192.168.1.x (dari Wi-Fi router yang berbeda). Paket data TCP/IP di lapisan jaringan lokal tidak dapat merutekan komunikasi langsung lintas subnet yang berbeda tanpa adanya perangkat perute (*router*) atau *gateway* yang mengaturnya.

2. Langkah teknis agar kedua perangkat berada dalam subnet yang sama:
    - Menyamakan sumber koneksi internet dengan cara menghubungkan laptop dan NodeMCU ke jaringan Wi-Fi / *hotspot* seluler yang persis sama (misalnya keduanya menggunakan *Tethering* dari HP yang sama, sehingga mendapatkan rentang IP yang se-subnet, contohnya 192.168.43.x).
    - Memastikan aturan pembukaan port (*Inbound Rules*) untuk port 1883 pada *Windows Defender Firewall* laptop telah aktif agar mengizinkan akses masuk dari jaringan privat/publik tersebut.

- **Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):**
Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0°C(misal: 31.05°C → 30.95°C → 31.02°C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
Rancang modifikasi alur Node-RED menggunakan **function node** berbahasa JavaScript yang menerapkan logika **Histeresis**:
    - LED baru menyala jika suhu >31.5∘C (msg.payload = {"led": 1}).
    - LED baru padam jika suhu mendingin hingga <30.0∘C (msg.payload = {"led": 0}).

    Tuliskan kode JavaScript yang Anda masukkan ke dalam Function Node tersebut!
Jawaban:



Pada sistem kendali dua posisi (*Bang-Bang Control*) standar, ketika suhu berada di sekitar titik ambang batas kritis (misalnya 31.0°C) dan mengalami fluktuasi kecil yang cepat—seperti perubahan nilai dari 31.05°C → 30.95°C → 31.02°C—aktuator (lampu LED) akan merespons dengan cara menyala dan padam secara beruntun dalam waktu singkat. Fenomena ini disebut sebagai **kedipan cepat (*flickering* atau *hunting*)**, yang dapat menurunkan umur komponen perangkat keras secara fisik.

- **Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):**
  Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.
    - Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?
    - Jelaskan perbedaan mendasar antara Bind Mount dan Named Volume pada Docker!

  Jawaban:

  1. **Apa yang terjadi jika baris volume dihapus dari docker-compose.yml lalu kontainer dimatikan dengan docker compose down?** Seluruh riwayat kanvas visual alur kerja (*flows*) pada Node-RED beserta penyimpanan data riwayat dan log Mosquitto akan hilang total (tidak persisten). Kontainer akan kembali ke kondisi kosong seperti saat pertama kali diunduh (*fresh image*) setiap kali kontainer dimatikan atau dihapus.
  2. **Perbedaan mendasar antara *Bind Mount* dan *Named Volume*:**
     - **Bind Mount:** Memetakan direktori atau berkas tertentu secara langsung dari sistem operasi laptop (*Host*) ke dalam kontainer (misalnya - ./nodered:/data). Pengguna dapat melihat dan menyunting berkasnya secara langsung dari luar kontainer**.**
     - **Named Volume:** Area penyimpanan khusus yang dikelola sepenuhnya oleh Docker di dalam ruang penyimpanan internal sistem (biasanya di dalam direktori sistem Docker), terisolasi dari struktur berkas utama pengguna di luar